

Table 3: Detection accuracy (at 5% FPR) of texts generated by GPT-4o and Gemini-1.5-Pro based on XSum dataset.

Model	Attack	Sim	Defense								Avg.
			LogRank	DetectGPT	DNA-GPT	Fast-DetectGPT	Raidar	TOCSIN	RoBERTa	R-Detect	
GPT-4o	No Attack	-	33.33	1.83	37.33	12.00	100.00	24.00	3.33	26.67	29.81
	Dipper	77.33	16.00	1.33	39.33	42.33	11.33	42.67	68.00	43.67	33.08
	Raidar	99.33	24.00	11.67	42.67	46.33	5.00	64.67	15.33	56.67	33.29
	Ours	96.67	3.00	0.67	22.00	6.33	5.00	10.00	16.67	4.67	8.54
Gemini-1.5-Pro	No Attack	-	21.67	13.00	36.00	32.00	28.00	33.33	12.67	24.67	25.17
	Dipper	80.67	11.00	0.67	23.33	51.17	5.00	48.00	72.67	35.67	30.94
	Raidar	100.00	31.00	8.67	26.00	39.83	30.00	48.67	22.67	42.67	31.19
	Ours	93.33	2.00	2.67	6.67	10.00	2.00	10.00	9.33	2.00	5.58

Table 4: Comparison of different paraphrasing attacks against 8 text-detection algorithms (at 1% FPR) using GPT-3.5-turbo generated texts from three different datasets. The best performances are bolded.

Dataset	Attack	Sim	Defense								Avg.
			LogRank	DetectGPT	DNA-GPT	Fast-DetectGPT	Raidar	TOCSIN	RoBERTa	R-Detect	
XSum	No Attack	-	32.44	6.00	41.33	83.00	5.75	94.67	44.67	49.33	44.65
	Dipper	86.67	7.33	0.00	8.00	44.67	2.75	46.00	74.00	28.33	26.39
	Raidar	100.00	21.00	1.00	5.33	57.33	3.25	85.33	39.33	58.67	33.91
	Ours	94.00	2.00	0.01	5.33	4.67	0.00	18.00	9.33	0.00	4.92
SQuAD	No Attack	-	20.50	2.67	0.00	82.33	4.44	83.33	15.33	58.00	33.33
	Dipper	75.33	3.00	1.33	1.33	57.67	1.00	50.00	39.33	43.33	24.62
	Raidar	95.33	21.33	5.33	0.00	59.33	5.20	68.00	8.67	48.67	27.07
	Ours	88.67	1.17	1.33	0.00	9.67	1.00	12.67	1.33	0.00	3.40
LongQA	No Attack	-	28.50	7.50	0.00	74.00	7.25	82.67	28.67	74.33	37.87
	Dipper	94.67	6.50	0.00	0.00	55.67	1.00	31.33	49.33	63.33	25.90
	Raidar	100.00	15.83	2.17	0.00	48.67	6.75	64.00	15.33	70.33	27.89
	Ours	95.33	2.67	1.33	0.00	5.33	0.00	8.67	2.00	3.00	2.88

(2) For machine-style distributions:

To elicit machine-like responses from LLMs, our prompt engineering (PE) constructs a wide range of prompts, *e.g.*, ask for direct paraphrasing or instruct the LLM to reply in the tone of an AI assistant. The LLM likelihood provided by a detector is used as a proxy to quantify the degree of machine-style characteristics in the output. We find that naively prompting an LLM to rewrite a machine text often reduces existing machine-related features, as the rewritten texts inherit mixed stylistic and lexical preferences from multiple LLMs, increasing textual diversity. To preserve machine-related features, we incorporate stylistic cues into the prompts, *e.g.*, instructions like "... in the tone of an AI assistant ..." or "... in a machine-writting style ..." to guide the LLM for a more typical machine expression, which indeed makes the output more easily detectable and, in turn, enhances the attack.

System Prompt

You are a helpful assistant.

User Prompt:

Repeat the following paragraph:

Considering that the original LLM-generated sentence carries the richest machine-related characteristics that are most easily detected, we adopt a direct and effective strategy by making LLM repeat the input sentence to obtain the most machine-like token probabilities. We provide the results of representative prompts in Fig. 3, which validate the effectiveness of our selection strategy.

C Additional Results

More Source LLMs. We provide the performance of paraphrasing on machine texts generated by GPT-4o (Achiam et al., 2023) and Gemini-1.5 Pro (Team et al., 2024) in Table 3. It can be observed that the proposed CoPA continues to achieve better attack effectiveness than existing paraphrasing methods. Also, the detection algorithms obtain relatively worse clean performance on texts generated by two advanced LLMs.

Results of FPR=1%. We then evaluate the at-

Table 5: Attack results paraphrased by different off-the-shelf LLMs (at 5% FPR) using GPT-3.5-turbo generated texts from three different datasets.

Dataset	Detector	Paraphraser				
		No Attack	R1-Distill-32B	QwQ-32B	GLM-4-9b-hf	Qwen2.5-72B
XSum	Fast-DetectGPT	95.33	24.33	11.33	31.33	17.00
	TOCSIN	98.00	42.67	10.00	87.00	26.67
	R-Detect	49.33	6.33	00.00	38.33	0.00
SQuAD	Fast-DetectGPT	93.50	26.83	9.50	31.33	27.50
	TOCSIN	92.67	42.00	17.33	81.33	25.33
	R-Detect	58.00	13.00	0.00	39.67	0.00
LongQA	Fast-DetectGPT	86.00	14.67	11.33	27.33	11.33
	TOCSIN	88.67	26.00	2.00	78.00	16.00
	R-Detect	74.33	22.67	0.00	47.33	3.00

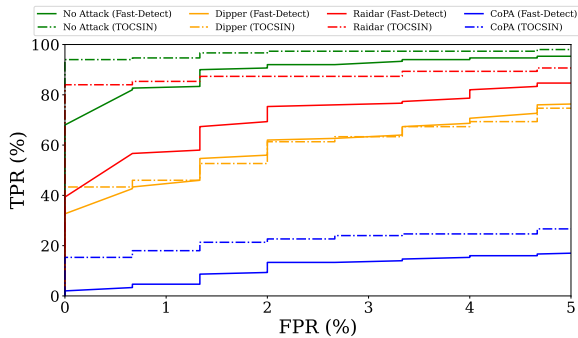


Figure 8: ROC (0-5% FPR) for GPT-3.5-turbo on Fast-DetectGPT and TOCSIN before and after paraphrasing. The proposed CoPA achieves the best detection rate across various FPRs.

tack under a more strict setup where FPR=1%. As shown in Table 4, CoPA consistently exhibits superior attack performance over current paraphrasing attacks. We also observe that some detection algorithms fail to produce any defense effects even without any attack at TPR=1%, raising concerns about their feasibility in practical scenarios.

ROC Curve Analysis. Figure 8 shows the TPR trends corresponding to different FPR values varying from 0% to 5%. As observed, TPR generally increases as FPR grows, and CoPA significantly reduces the TPR values of detection algorithms across all FPR thresholds, which strongly validates the effectiveness of the proposed CoPA. Note that under the stricter and more realistic setting of FPR = 1%, CoPA reaches a detection accuracy below 20% against these defenses, further underscoring its superior performance.

Impact of LLM Sampling Parameters. During decoding, LLMs employ various sampling parameters such as Top- p , Top- k , and the temperature coefficient T to adjust the sampling results.

To investigate their influence, we conduct ablation studies regarding these parameters during the decoding process of our paraphrasing in Figure 9. For Top- p and Top- k , the reduction of p or k results in a drop in sampling diversity since fewer tokens are retained, thereby generating more machine patterns and impairing the performance. For the temperature (T), numeric results indicate that the increase of T facilitates the creativity and diversity of sampling choices by reducing the difference in token probabilities, hence better misleading the detectors. An adversary can adjust these parameters based on their needs to achieve a superior attack while controlling the writing styles of generated sentences.

Results of More LLMs as paraphraser. To validate the universality of the proposed attack, we next consider more LLMs as the paraphraser, including various model scales and recently prevalent reasoning-based models. Specifically, we consider Deepseek R1-Distill-32B (DeepSeek-AI, 2025), QwQ-32B (Team, 2025), and GLM-4-9B-hf (GLM, 2024). The quantitative results in Table 5 show the effectiveness of the proposed CoPA across various LLMs. Note that the QwQ-32B generally achieves the best performance. However, the reasoning-based models require significantly more inference time than regular models. We choose the Qwen2.5-72B to balance effectiveness and efficiency.

Table 6: Comparison of CoPA with a surrogate-based paraphrasing attack against two SOTA detectors.

Attack	Sim	Defense	
		Fast-DetectGPT	TOCSIN
RedTeaming	78.00	30.00	38.67
Ours	94.00	17.00	26.67

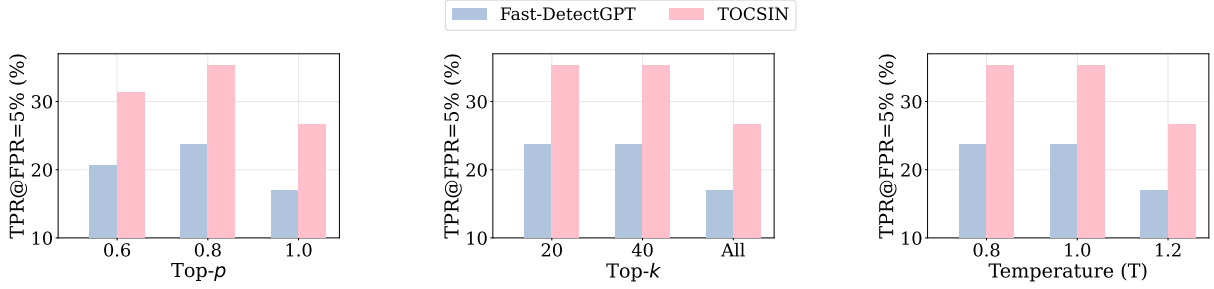


Figure 9: Detection accuracy (at FPR=5%) of CoPA under different sampling parameters against Fast-DetectGPT (Bao et al., 2024) and TOCSIN (Ma and Wang, 2024). We calculate results using samples from the XSum dataset.

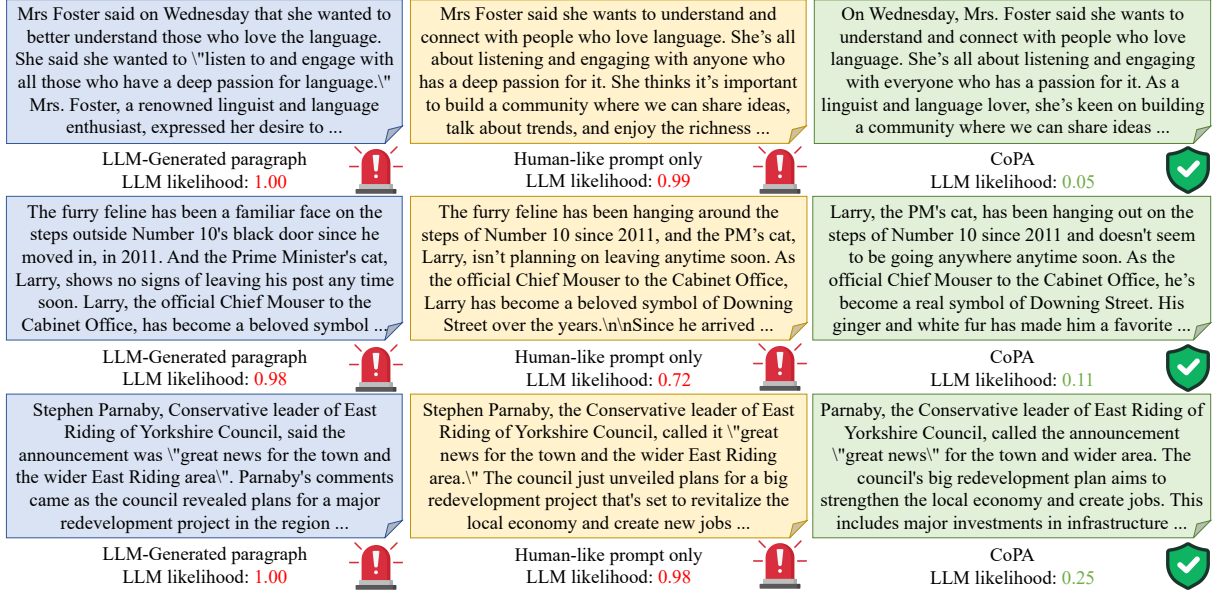


Figure 10: Visualization of paraphrased sentences from human-like distribution p_h and our contrastive distribution p_c (i.e., our CoPA). The LLM likelihood is calculated based on Fast-DetectGPT.

Comparison with a surrogate-based baseline. This paper follows Dipper (Krishna et al., 2023) and focuses on the more practical and universal attacks without relying on any surrogate detection model. A direct comparison of these methods with the surrogate-based paraphrasing attack introduced in RedTeaming (Shi et al., 2024) may raise concerns of unfairness. However, results in Table 6 reveal that the proposed CoPA can still achieve better performance than RedTeaming.

Notably, we include these results only for experimental completeness. The surrogate-based methods are not the focus of this work.

Ablation study of the adaptive truncation mechanism. To avoid penalizing the probabilities of reasonable and valid tokens, we incorporate an adaptive truncation mechanism to constrain the output token distribution. We provide an ablation study to investigate its influence. As observed in Table 7, by truncating the token distribution within

a reliable token candidate pool, we can improve the text quality of the generated sentences while maintaining attack effectiveness.

Table 7: Ablation on the adaptive truncation technique.

Method	Sim	Fast-DetectGPT	TOCSIN	R-Detect
w/o truncation	93.33	19.67	31.33	5.33
CoPA	94.00	17.00	26.67	4.67

D Analysis of human-like prompt only

As shown in Figure 10, we observe that solely relying on the human-like prompt x_h results in unstable attack performance, i.e., some sentences derived from the human-like distribution p_h still retain prominent machine-related features, which render them easily identifiable by text detectors. To alleviate this issue, our CoPA framework utilizes an auxiliary machine-like distribution to fully remove these machine characteristics from p_h , significantly